Metropolia

Digital Technology

Microcontroller Project

Edvárd Nagy, Máté Mirkovszki, Alasdair Johnstone

Contents

Project Overview - 1
Introduction - 2
System Architecture - 3
System Implementation – 4
Design Process – 4.0
Modular Construction & Testing – 4.1
Component Level Assembly – 4.2
Final Intergrated System – 4.3
Results - 5
Conclusion - 6
References - 7
Learnings - 8
Feedback - 9

Project Outline

This project covers the design, construction, and successful demonstration of a simple 2-bit microcontroller built entirely from basic digital logic components. The main goal was to create a self-contained circuit capable of running autonomously. The key challenge was integrating logic gates, flip-flops, registers and memory into one system that could execute a pre-loaded program without any manual input—essentially emulating a microcontroller.

Introduction

The goal of this project was to move beyond theoretical concepts of computer architecture and digital logic into practical, hands on implementation. By constructing a microcontroller from its most basic components, the project provides foundational insights between hardware and software at the lowest level. This process reinforces the fundamental principles of digital circuit design, data path organization, and instruction cycle execution, bridging the gap between block diagrams and functioning hardware.

A complete 2-bit hardware microcontroller was successfully implemented across several circuit bread boards. The implementation adhered to the project constraints, utilizing only approved components such as standard logic gates, multiplexers, encoders, and SRAM. The final constructed device successfully executed a pre-loaded test program from memory, driven solely by the clock signal. This outcome validated the architectural design, the logic of the control unit, and the physical integrity of the circuit implementation, demonstrating a comprehensive success.

System Architecture

The selection of components was guided by the project requirements to build a complete system from discrete digital building blocks. Each component serves a distinct and critical function within the overall architecture.

• 2-bit Arithmetic Logic Unit (**ALU**): The computational core of the microcontroller, designed specifically to perform four required operations (ADD, SUB, AND, OR) on two 2-bit operands.

• **Registers**: A set of 2-bit storage elements built from flip-flops. These registers are used to hold operands for the ALU, store intermediate results, and temporarily hold the current instruction fetched from memory in the Instruction Register (IR).

• **SRAM**: Static Random-Access Memory serves as the memory system for the microcontroller, storing both the program instructions (opcodes) and the data values used during program execution.

• Program Counter (**PC**): A dedicated counter circuit that holds the memory address of the next instruction to be executed. It automatically increments with each clock cycle, enabling the sequential execution of the program stored in SRAM.

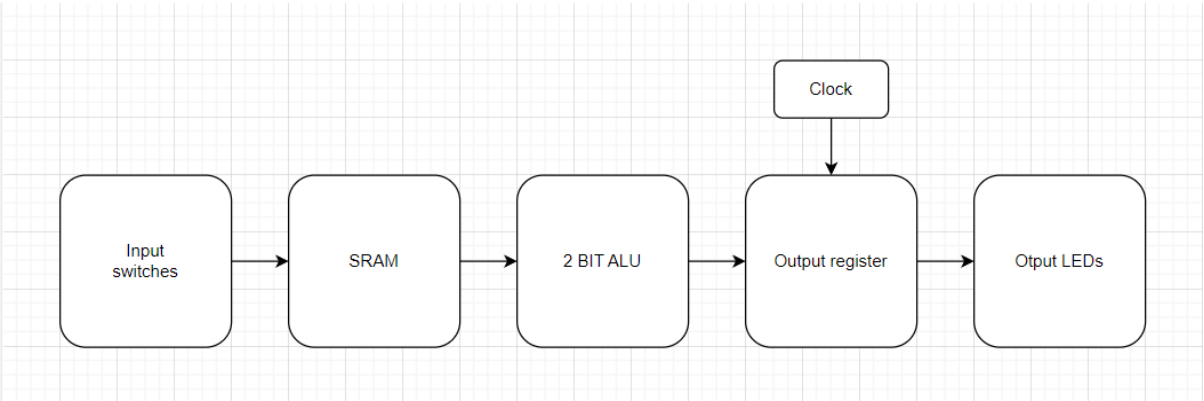
• **Control Unit**: The logical "brain" of the system, composed of decoders and multiplexers. It interprets the opcode held in the Instruction Register and generates the necessary control signals to manage data flow between the registers, ALU, and memory, orchestrating the entire fetch-decode-execute cycle.
• **NE555 Timer** (Clock): An oscillator circuit built around an NE555 timer IC. It generates the stable, low-frequency clock pulses that synchronize all operations within the microcontroller, ensuring that state changes occur in an orderly and predictable manner.

List of the component used:

7404	–	2 gate OR
7408	–	2 gate AND used for AND op and adder carries
7432	–	2 gate OR used for OR op and carry merges
7486	–	2 gate XOR
74LS373	–	D-Latch used for SRAM register
74LS175	–	D-flipflip Output Register
74LS153	-	Multiplexer

used as a decoder

NE555	-	Timer Clock Keeps the ALU combinational. Drives the registers output in the clock´s rising edges at ~1 Hz.



Truth tables:

	F1/F0
ADD	00
SUB	01
OR	10
AND	11

AND (f1 f0 = 11)			OR (f1 f0 = 10)			SUB (f1 f0 = 01)				ADD (f1 f0 = 00)			
A1A0	B1B0	Result C1C0	A1A0	B1B0	Result C1C0	A1A0	B1B0	Result C1C0	OW	A1A0	B1B0	Result C1C0	OW
00	00	00	00	00	00	00	00	00	0	00	00	00	0
00	01	00	00	01	01	00	01	11	1	00	01	01	0
00	10	00	00	10	10	00	10	10	1*	00	10	10	0
00	11	00	00	11	11	00	11	01	1	00	11	11	0
01	00	00	01	00	01	01	00	01	0	01	00	01	0
01	01	01	01	01	01	01	01	00	0	01	01	10	1*
01	10	00	01	10	11	01	10	11	1*	01	10	11	0
01	11	01	01	11	11	01	11	10	1*	01	11	00	1
10	00	00	10	00	10	10	00	10	0	10	00	10	0
10	01	00	10	01	11	10	01	01	0	10	01	11	0
10	10	10	10	10	10	10	10	00	0	10	10	00	1*
10	11	10	10	11	11	10	11	11	1	10	11	01	1*
11	00	00	11	00	11	11	00	11	0	11	00	11	0
11	01	01	11	01	11	11	01	10	0	11	01	00	1*
11	10	10	11	10	11	11	10	01	1*	11	10	01	1*
11	11	11	11	11	11	11	11	00	0	11	11	10	1

System Implementation

4.0 Design Process

We started by defining the instruction set and designing each subsystem ALU, register, memory, control unit, and clock. For the ALU, we designed a 1-bit ALU in Logisim, with the required operations in mind, then connected it to another 1-bit ALU to create the 2-bit ALU. The Logisim design had to be converted to a Multisim design. For the ALU Once each schematic was finalized in Multisim we could then move on to the next steps.

4.1 Modular Construction and Testing

A phased, modular approach was adopted for the hardware assembly. Each major functional block of the microcontroller was first constructed and tested as an independent unit on a breadboard. This included the NE555 timer clock, the 2-bit ALU, individual decoders, and so on. This methodology proved critical for effective debugging, as it allowed for the verification and isolation of any wiring or logic errors within a specific module before it was integrated into the larger system. By confirming the correct operation of each part individually, the complexity of debugging the final, fully assembled device was significantly reduced.

4.2 Component-Level Assembly

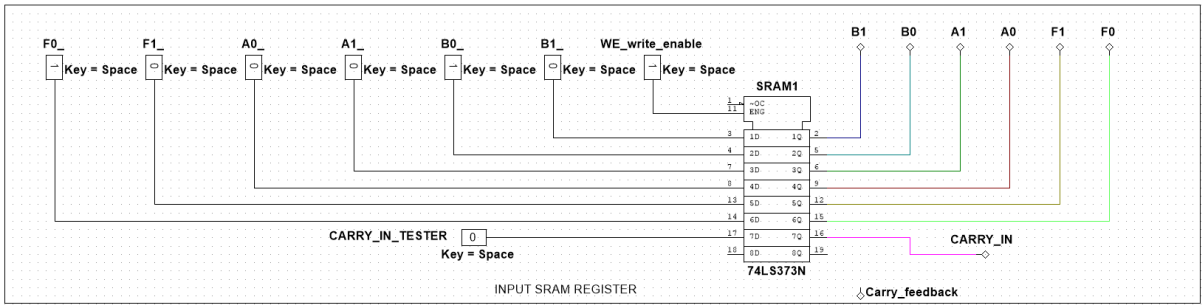
The key modules were constructed using standard, off-the-shelf integrated components. The core of the ALU's arithmetic circuits, the half-adders and full-adders, were built from fundamental logic gates. Specifically, components such as the 7486 (4 channel 2-Input XOR) were fundamental for generating the sum bits in the half and full-adders, while 7408 (4 channel 2-Input AND) gates were used to compute the carry logic. This component-level assembly provides a granular understanding of how complex functions are derived from simple logical operations.

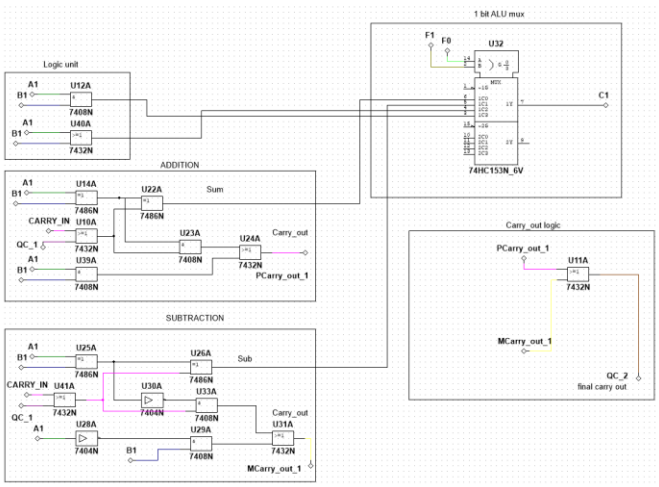
4.3 Final Integrated System

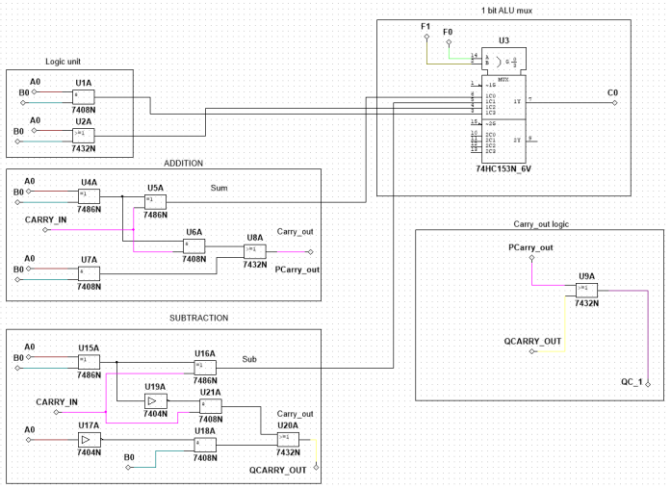
Following the successful verification of all individual modules, the final assembly phase began. All components—the clock, ALU, register, decoder, control logic, and SRAM—were carefully wired together and integrated onto a single, compact circuit board. The

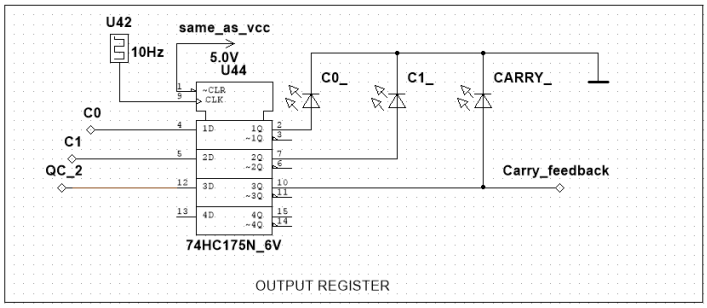
result was a stand-alone device that operates autonomously, executing instructions sequentially under the control of the NE555 clock signal. This final integrated system fulfilled the primary project requirement of creating a self-contained microcontroller that functions without external manipulation of its internal states.
With the hardware fully integrated and bench-tested, the project advanced to its final and most critical phase: executing a comprehensive test program to validate the end-to-end functionality of the design.

Schematics:









Results

The demonstration was initiated by starting the NE555 clock, which was configured to run at a low frequency (10 Hz). This slow clock speed allowed for visual observation of the program's execution in real-time. A series of LEDs were connected to the outputs of key register and the ALU. As the microcontroller stepped through the program, the state of these LEDs changed with each clock cycle, providing a direct visual representation of the data being processed. This method allowed for the

verification of the output at each step of the program, confirming that each instruction was being fetched, decoded, and executed correctly.
Our circuit's functions worked pretty well however we traced an issue to the OR function which caused us functionality issues we were unable to trace down completely. Additionally the carry in is a function which worked in MultiSim created issues when we put our circuits components together.

Conclusion

This project successfully achieved its primary objective: the design, implementation, and validation of a functional 2-bit microcontroller using only discrete digital logic components. The final device, built as a stand-alone system on a single circuit board, demonstrated its capability to autonomously fetch and execute a sequence of instructions stored in memory. The testing of its operation was confirmed through a test program that verified its arithmetic and logical functions.
The project effectively met its core educational and technical goals spot on. It provided a practical, hands-on experience that bridged the gap between the theoretical principles of computer architecture and the realities of hardware engineering. The process of building a processor from the ground up offered deep insights into the fundamentals of basic modern computing.

References

Maria was a huge help while building the circuit assisting us with our questions we had about the difference between our MultiSim circuit and our physical circuit. She was always very eager to help and she would often ask if anybody in the room had any last questions before heading off for the day.

"A Novel Approach to Design 2-bit Binary Arithmetic Logic Unit (ALU) Circuit Using Optimized 8:1 Multiplexer with Reversible logic" by Vandana Shukla, O. P. Singh, G. R. Mishra, and R. K. Tiwari

Design and simulation of Arithmetic Log Unit
https://he-coep.vlabs.ac.in/exp/arithmetic-logic-unit/theory.html

2 bit ALU circuit diagram
https://www.gadgetronicx.com/2-bit-alu-circuit-design/

4bit ALU design
https://pages.cs.wisc.edu/~kzhao32/projects/ece467_4bitALU.pdf?

8 bit computer design
https://eater.net/8bit/alu

Learnings

We first simulated the circuit in MultiSim and everything worked smoothly there, however we had serval issues going from MultiSim to physical build. One issue was that the logic gates signal would not open at 5mV but instead needed 1v/5v to operate. We got around these issues with the weak signal by rearranging the circuit.
Another issue we came across going from MultiSim to the physical build was with floating parameters of the circuit. In MultiSim not all the operating pins need to be connected. This is just a simple quirk of MultiSim to make it easier and cleaner to build circuits there, which is an easy fix on the physical build by grounding the not needed pins. We originally wanted more LEDs to signal where the process is currently but due to our limited current source it was not possible, our set safely chosen 200mA current can only supply a couple of LEDs at a bright level. When building on a breadboard it's incredibly easy to accidentally place a wired one hole off and for you not to notice until testing, and then it's very difficult to trace back to that location so we found it crucial to label many of the wires to help with trouble shooting later.

• The project offered a profound appreciation for the practical application of Boolean logic and digital design principles. Designing and building complex functional units like the ALU from basic gates provides a tangible understanding of how simple true/false logic scales are to perform complex computation.
• The critical importance of a systematic, modular build-and-test strategy was a major lesson. Assembling and verifying each component independently was essential for managing complexity and simplifying the debugging process. This approach prevented the overwhelming task of troubleshooting the entire integrated system at once.

Valuable experience was acquired in hardware debugging. The process involved identifying and resolving a range of real-world issues, including subtle timing problems, incorrect wiring, and logic errors that were not apparent in the paper design. This hands-on troubleshooting is an essential skill for any hardware engineer.
There were definitely some problem-solving skills which were sharpened during this project. Trying to locate where issues lie with the circuit first in MultiSim then on your physical build takes a lot of time and effort. We learnt a decent amount about creating a microcontroller, but this project was very time-consuming and difficult for the points.

Feedback

Breadboard ground & Vcc connections caused issues with our circuits – traced problems back to there. Possible lose connections within the breadboard too – For example, there could be issues with our circuit but after a small break we would come

back to find the circuit was working fine – we suspected it could be due to lose connections in the board. This was quite an advanced project and I believe the whole class felt like it was quite a jump from what we have done previously.